

**PROJECT SCORING, EVALUATION &
MANAGEMENT SYSTEM (PSEMS)**
CO3554 DATA MANAGEMENT PROJECT

NAME : K.T.D.H. NIMSARA
INDEX NO : 22/ENG/082
REGISTRATION NO : EN108960
SUBMISSION DATE : 30/03/2026

Table of Contents

Table of Contents.....	2
1. Introduction.....	
2. Problem Statement & Objectives.....	4
Problem Statement.....	4
Project Objectives.....	4
Platform Objectives	4
Data Mining & ML Objectives	5
Technical Objectives.....	5
3. Literature Review	6
Academic Project Management & LMS Onboarding.....	6
Preference-Based Matching in Academic Allocation.....	6
Educational Data Mining & Performance Prediction	6
Text Similarity, Plagiarism Detection & Semantic Embeddings.....	6
4. Dataset Overview.....	8
Data Sources	8
Student Email Database & Account Provisioning	8
Historical Project Dataset (ML Training).....	9
Expected Data Volume	9
Data Preprocessing & Privacy	9
5. Proposed System Architecture.....	10
Complete Project Lifecycle Flow	10
System Architecture.....	12
Role-Based Access Control (RBAC) Hierarchy.....	13
6. Tools and Technologies.....	14
Backend & Database.....	14
Machine Learning Stack.....	14
Frontend & Infrastructure	15
7. Project Timeline.....	16
8. Expected Outcomes	18
System Deliverables	18
Expected Benefits per Stakeholder	19
9. Conclusion	19
10. References.....	20

Abstract

PSEMS (Project Scoring, Evaluation & Management System), a centralized medium-sized enterprise-grade online platform created for module CO3554, is presented in this proposal. With the help of a configurable timeline engine and a structured role hierarchy, PSEMS oversees the entire academic project lifecycle, from the creation of course project instances to the final aggregated findings. Similar to conventional onboarding, student accounts are automatically provisioned by the System Admin using university email records and pre-assigned temporary passwords. The system administrator approves lecturers who self-register. The entire CPI lifecycle is managed by the course coordinator, who also sets phase timelines, adds supervisors, manages idea announcements, oversees project selection using an Expression of Interest and Mutual Confirmation mechanism, configures fully customizable rubric-based evaluations, schedules assessments, and views a final analytical summary.

Beyond platform management, PSEMS integrates a Machine Learning and Data Mining layer leveraging historical faculty project data to deliver seven intelligent features: real-time idea suggestions, success rate prediction, similar project recommendation, topic trend clustering, supervisor compatibility scoring, grade distribution analytics, and a semantic plagiarism similarity warning system.

1. Introduction

A number of project-based evaluations, such as Final Year Projects (FYP), Data Management Projects, High Performance Computing (HPC) Mini Projects, and Innovation Challenges, are used by universities to show applied technical skills. Course organizers, supervisors, and evaluators have a huge burden of paperwork as a result of these assessments, which span several departments, cohorts, and academic years.

The procedures controlling academic project management are still ineffective in most of universities, despite their institutional importance. Manual spreadsheet imports are necessary for student onboarding. There is no systematic mutual-agreement process for supervisor allocation. Cross-evaluator verification and organized rubric enforcement are absent from evaluation procedures. Most importantly, the wealth of historical data produced over years of project evaluations is never examined or reused to help future coordinators and students.

By automating student provisioning, formalizing supervisor-student matching through a preference-based mutual confirmation model, layering machine learning intelligence on top of historical project data to provide real-time guidance throughout the process, and putting the course coordinator at the center of the project lifecycle with full configurable control over every phase, PSEMS overcomes these issues.

2. Problem Statement & Objectives

Problem Statement

Problem	Description	Impact
Manual & Fragmented Student Onboarding	Student accounts are created manually without systematic email provisioning. No centralized mechanism exists to onboard an entire year's cohort across departments the way an LMS does.	Administrative bottleneck; students miss early project phases
Unstructured Supervisor-Student Matching	No formal mechanism for students to express preferences for supervisor projects or for supervisors to accept/decline groups. Conflicts between supervisors wanting the same group are unresolved.	Unfair allocation, supervisor overload, student dissatisfaction
Inconsistent & Opaque Evaluation	No structured rubric-based digital evaluation; no cross-evaluator verification layer. Students have no transparent view of how marks were calculated.	Unfair grading outcomes, academic integrity risk
Unused Historical Project Data	Years of project data idea titles, descriptions, grades, supervisor assignments are never analyzed. Students receive no guidance on idea originality or project viability.	Missed opportunity for data-driven academic improvement

Project Objectives

Platform Objectives

- Develop a centralized multi-tenant platform where the Course Coordinator creates and fully controls Course Project Instances (CPIs) defining the project type (FYP, DM, HPC), timeline phases, supervisors, and evaluation structure.
- Implement automated student account provisioning: System Admin holds university email records per department and year, auto-creates accounts, and dispatches temporary password emails requiring change on first login.
- Support lecturer self-registration with System Admin approval, enabling lecturers to be assigned as supervisors, evaluators, or head judge within any CPI by the Course Coordinator.
- Support two CPI management modes determined automatically at CPI creation: Supervisor-Led mode (supervisors added by coordinator full EOI + Mutual Confirmation flow applies) and Coordinator-Managed mode (no supervisors added coordinator posts ideas directly, approves or rejects student ideas, and finalizes all allocations without a supervisor layer).
- Build an Expression of Interest and Mutual Confirmation project selection mechanism for Supervisor-Led CPIs, allowing students to rank preferred projects and supervisors to accept or decline groups, with coordinator override authority at all times.
- Implement a fully customizable rubric-based evaluation system where the Course Coordinator defines stages, weights, and criteria; evaluators control the specifics within their assigned session; and the Head Judge verifies and finalizes marks before they reach the coordinator.
- Provide the Course Coordinator with a full analytical summary at project end: individual marks per student/group, aggregated percentage-weighted scores, grade distributions, and evaluator consistency metrics.

Data Mining & ML Objectives

- Train a real-time idea suggestion engine using TF-IDF or Sentence-BERT embeddings on historical project data to assist students during idea formulation.
- Develop a project success rate prediction model using historical evaluation outcomes and project features, handling both supervised and coordinator-managed historical projects supervisor presence treated as a nullable feature rather than a required one.
- Implement a semantic plagiarism and similarity warning system using vector embeddings and cosine similarity to detect conceptually similar ideas across active and historical batches.
- Build a topic trend clustering dashboard and grade distribution analytics module for coordinator insight, including comparison of grade outcomes between supervised and coordinator-managed project batches.
- Implement supervisor compatibility scoring as a conditional ML feature displayed only in Supervisor-Led CPIs where a supervisor exists to match against.

Technical Objectives

- Design a scalable PostgreSQL schema with pgvector extension for vector similarity search, supporting thousands of concurrent student records across multiple departments and years.
- Implement the ML layer as a decoupled Python FastAPI microservice ensuring core platform stability is independent of ML availability.

3. Literature Review

Academic Project Management & LMS Onboarding

Al-Ajlan and Zedan examined Learning Management System adoption in academic environments and found that platforms such as Moodle effectively handle content delivery and centralized student provisioning including bulk account creation with email dispatch but offer no support for structured project lifecycle management, configurable rubric evaluation, or supervisor-student allocation. Siau and Rossi demonstrated through a review of academic evaluation techniques that rubric-based criterion-referenced assessment yields significantly more reliable and reproducible grading outcomes than holistic evaluation. These findings jointly establish two gaps directly addressed by PSEMS: the need for LMS-grade automated student onboarding integrated with a project management platform, and the need for structured rubric enforcement with a verification layer.

Preference-Based Matching in Academic Allocation

Gale and Shapley introduced the deferred acceptance algorithm for stable matching between two groups with ranked preferences, which has been widely applied in academic supervisor-student allocation. However, Irving et al. noted that in practice, academic project allocation involves significant asymmetries supervisors have capacity constraints, students may have equal preferences across projects, and coordinators require override authority making strict algorithmic matching suboptimal for real-world deployment. Their work recommends hybrid systems combining expressed preferences with human coordinator judgment, which is precisely the model adopted in PSEMS's Expression of Interest and Mutual Confirmation mechanism. This approach preserves the preference signal of both students and supervisors while keeping the coordinator in control of conflict resolution and edge cases.

Educational Data Mining & Performance Prediction

Romero and Ventura defined educational data mining (EDM) as the application of data mining techniques to educational data, identifying prediction, clustering, and relationship mining as primary techniques. Baker and Yacef noted EDM's trajectory toward predictive systems applied to longitudinal cross-batch data precisely the direction PSEMS pursues with its ML layer. Amrieh et al. applied Random Forest classifiers to educational datasets, achieving 67–75% cross-validated accuracy in student performance prediction, validating the ensemble approach used in PSEMS's success predictor. No prior EDM work applies these techniques to cross-batch project-level data for real-time guidance at the idea submission stage

Text Similarity, Plagiarism Detection & Semantic Embeddings

Alzahrani et al. demonstrated that character and token-based plagiarism detection methods are insufficient for paraphrase plagiarism, recommending semantic vector-space approaches. Salton and Buckley's foundational TF-IDF framework handles keyword similarity but cannot detect semantically equivalent ideas expressed with different vocabulary. Devlin et al. introduced BERT, whose contextual representations capture semantic meaning with significantly greater fidelity than bag-of-words models.

Reimers and Gurevych extended this with Sentence-BERT, producing semantically meaningful sentence embeddings with efficient cosine similarity computation reducing pairwise comparison to a single embedding step, making real-time similarity search across thousands of project ideas practical.

Academic Information Systems in Sri Lankan Universities

Systems such as the Project Supervision and Evaluation Management System (PSEMS) are essential in Sri Lankan universities. The majority of universities use Moodle as their LMS, according to a study by Thuseethan, Achchuthan, and Kuhanesan. While Moodle enables student registration and content distribution, it lacks the kind of tools for supervisor-student allocation, organized project lifecycle management, and rubric-based evaluation. These weaknesses highlight the importance of systems like PSEMS.

Thuseethan's additional investigation revealed that many university departments still rely on manual procedures, which causes delays, increased error rates, and trouble obtaining information. This led to the proposal of a Department Management System. The study showed that organized management systems greatly increase administrative task accuracy and efficiency.

Furthermore, students are prepared for more collaborative academic platforms, according to study by Gunawardhana . conducted at the Universities of Colombo and Moratuwa. Students highlighted a need for collaboration tools, peer engagement, and group work environments all of which are lacking in the current Moodle systems. These results validate the group-based features and collaborative project management suggested in PSEMS.

Summary of Gaps Addressed

LMS-grade student provisioning linked with project management is absent from current academic systems. In particular, Moodle-based LMS platforms used by Sri Lankan universities lack project lifecycle management, rubric evaluation, and supervisor assignment. In academic settings, hybrid preference-plus-coordinator models are more suitable than pure algorithmic allocation. Cross-batch project data for real-time student guidance has not been subjected to Educational Data Mining. Academic concept submission methods do not include semantic similarity checks. All of the shortcomings that have been found are addressed by PSEMS, with a focus on Sri Lankan higher education.

4. Dataset Overview

Data Sources

Source	Type	Provider	Purpose
University Student Email Records	Structured per department & year	System Admin / University Registry	Automated student account provisioning
Faculty Historical Project Data	Offline CSV/Excel	Faculty / Department	Train all ML models, build vector similarity index
Live Platform Operational Data	Real-time	PSEMS users	Core platform functionality, live analytics
Student File Submissions	Binary (PDF, DOCX)	Students	Proposals, reports, presentations via Supabase Storage

Student Email Database & Account Provisioning

The System Admin maintains the university student email database, structured by department and academic year (e.g., 1st Year Computer Engineering, 2nd Year Computer Engineering). This database is used to automatically provision student accounts at the start of each semester:

1. System Admin filters students by department and year cohort
2. System Admin triggers bulk account creation the system generates accounts with temporary passwords
3. Automated emails dispatched to each student's university email with login credentials
4. On first login, students are forced to change their password before accessing any features
5. Students are then visible in the system and can be enrolled into CPIs by the Course Coordinator

Email Database Attribute	Description
student_id	University-assigned unique student identifier
full_name	Student full name
university_email	Official university email address used for account creation
department	Department of enrollment (e.g., Computer Engineering, Computing)
academic_year	Current year of study (1st, 2nd, 3rd, 4th year)
program	Degree program (e.g., Software Engineering, Computer Science)
enrollment_year	Academic year of enrollment

Historical Project Dataset (ML Training)

The faculty provides anonymized historical project data from previous student batches. This forms the training foundation for all ML models

Attribute	Type	ML Usage
idea_title, idea_description	Text	Primary input for SBERT embeddings and TF-IDF vectorization
topic_category	Categorical	K-Means clustering; success predictor feature
batch_year	Integer	Temporal trend analysis across academic years
group_size	Integer	Success predictor feature
supervisor_id (nullable, anonymized)	String	Supervisor compatibility scoring (Supervisor-Led projects only). Rows without supervisor_id encoded as NO_SUPERVISOR category not dropped from training data. Enables comparison of supervised vs. coordinator-managed project outcomes.
proposal_score, mid_score, final_score	Float (0–100)	Success predictor features and partial targets
final_grade	Categorical (A–F)	Primary classification target for success predictor
outcome_status	Categorical	Secondary analysis: Completed / Incomplete / Withdrawn

Expected Data Volume

Data Entity	Estimated Scale	Storage
University student email records	5,000 – 15,000 per department batch	System Admin database
Historical project ideas (ML training)	500 – 2,000+ rows (min. 3 batches)	PostgreSQL + pgvector
Live student accounts	5,000 – 10,000 active	PostgreSQL
Live rubric scores (multi-semester)	50,000 – 200,000+	PostgreSQL
Idea vector embeddings	2,000 – 12,000 (384-dim)	pgvector / FAISS index
Audit log entries	500,000+	PostgreSQL archive schema

Data Preprocessing & Privacy

Historical faculty data undergoes cleaning (deduplication, missing value handling), text normalization (lowercasing, stopword removal via NLTK), label encoding, Sentence-BERT embedding generation, and an 80/20 stratified train/test split before model training. All historical data is fully anonymized student names, IDs, and emails are removed before ML use. University email records used for provisioning are never exposed to ML models or any user-facing feature.

5. Proposed System Architecture

Complete Project Lifecycle Flow

The following describes the end-to-end lifecycle of a Course Project Instance in PSEMS, from system setup to final results.

SYSTEM ADMIN	• Maintains university student email database (by department & year) • Bulk auto-creates student accounts and dispatches temporary password emails • Approves self-registered lecturers; assigns Course Coordinators to courses
STEP 1 - CPI Creation	Course Coordinator creates Course Project Instance defines project type (FYP / Data Management / HPC), department, academic year, and group or individual participation mode.
STEP 2 - Timeline Setup	Coordinator defines open/close dates for each phase: <i>Student Registration → Supervisor Addition (optional determines CPI mode) → Idea Announcement → Project Selection → Project Registration → Evaluation Config → Proposal Submission → Scheduling → Evaluation Execution → Final Submission</i>
STEP 3 - Supervisor Addition (Optional)	This step determines the CPI management mode automatically: If coordinator adds supervisors → SUPERVISOR-LED MODE: Supervisors post ideas, EOI + Mutual Confirmation flow applies, supervisor accept/decline mechanism active. If coordinator skips this step → COORDINATOR-MANAGED MODE: No supervisors. Coordinator posts ideas directly. Student ideas approved or rejected by coordinator. Coordinator handles all allocations. <i>Both modes share the same evaluation system, timeline structure, and all ML features except supervisor compatibility scoring.</i>
STEP 4 - Student Registration	Students (already provisioned with accounts) register for the CPI. They form groups or register individually depending on CPI configuration. Group leader invites members; invited students must accept. Groups lock when the registration window closes.
STEP 5 - Idea Announcement	SUPERVISOR-LED MODE: • Supervisors post project ideas visible to ALL enrolled students. • Student groups/individuals post own ideas visible ONLY to that group, all supervisors, and coordinator. NOT visible to other groups. COORDINATOR-MANAGED MODE: • Coordinator posts project ideas directly visible to ALL enrolled students. • Students can still post own ideas visible ONLY to that group and the coordinator. NOT visible to other groups. • Coordinator reviews all student-submitted ideas and Accepts or Rejects each one directly. <i>ML (both modes): Similarity warning triggers on idea submission. Suggestion engine assists while typing.</i>

STEP 6 - Project Selection	SUPERVISOR-LED MODE: • Expression of Interest: Students mark up to 3 supervisor projects (ranked 1–3). Students with own ideas mark 'Seeking Supervisor'. Supervisors browse student ideas and mark 'Willing to Supervise'. • Final Selection: Group makes one choice. Supervisor project selected → supervisor Accepts or Declines. Own idea selected → a willing supervisor must formally Accept. • Conflict (2 supervisors want same group): The group chooses their preferred supervisor. COORDINATOR-MANAGED MODE: • Students select from coordinator-posted ideas or submit own accepted ideas. No supervisor accept/decline step. Coordinator directly reviews and approves all group-project pairings. <i>ML (Supervisor-Led only): Supervisor compatibility score shown on project card. Success predictor shown in both modes.</i>
STEP 7 - Project Registration	Coordinator reviews the full allocation map for the CPI. In Supervisor-Led mode: resolves unmatched groups and unassigned supervisor projects. In Coordinator-Managed mode: reviews all pairings and confirms or reassigns. Coordinator can override any allocation in either mode. Once finalized, all allocations are locked and all parties notified.
STEP 8 - Evaluation Config	Coordinator defines evaluation stages with full customization: stage name, weight %, submission requirements, number of evaluators, rubric criteria and their weights. <i>Example: Proposal (15%) Mid Evaluation (25%) Final Demo (40%) Report (20%)</i> Coordinator assigns specific evaluators per stage. Head Judge designated from evaluator pool.
STEP 9 - Proposal Submission	Students upload proposal documents through the platform before the configured deadline. Submissions are stored in Supabase Storage and linked to the group's project record.
STEP 10 - Scheduling	Coordinator opens the availability submission window. Supervisors and evaluators submit available time slots via a calendar interface. Coordinator reviews, resolves conflicts, and finalizes the timetable. Students and evaluators receive automated notifications with their confirmed schedule.
STEP 11 - Evaluation	Evaluators submit rubric scores for assigned groups. Each evaluator can customize specifics within their assigned session. Evaluators cannot see each other's scores before the Head Judge review phase. Head Judge reviews all evaluator scores side-by-side with deviation flags. Head Judge approves or requests correction from specific evaluators. Finalized marks flow to the coordinator.
STEP 12 - Finalization	Marks aggregated automatically by configured stage weights. Each student's/group's mark shown with full breakdown per stage. Coordinator views full analytical summary. <i>ML: Topic trends, grade distribution analytics, evaluator consistency all visible in coordinator dashboard.</i> Coordinator publishes results students notified and can view final marks.

System Architecture

PSEMS follows a four-tier layered architecture. The ML microservice is deliberately decoupled; if unavailable, the core platform continues operating normally.

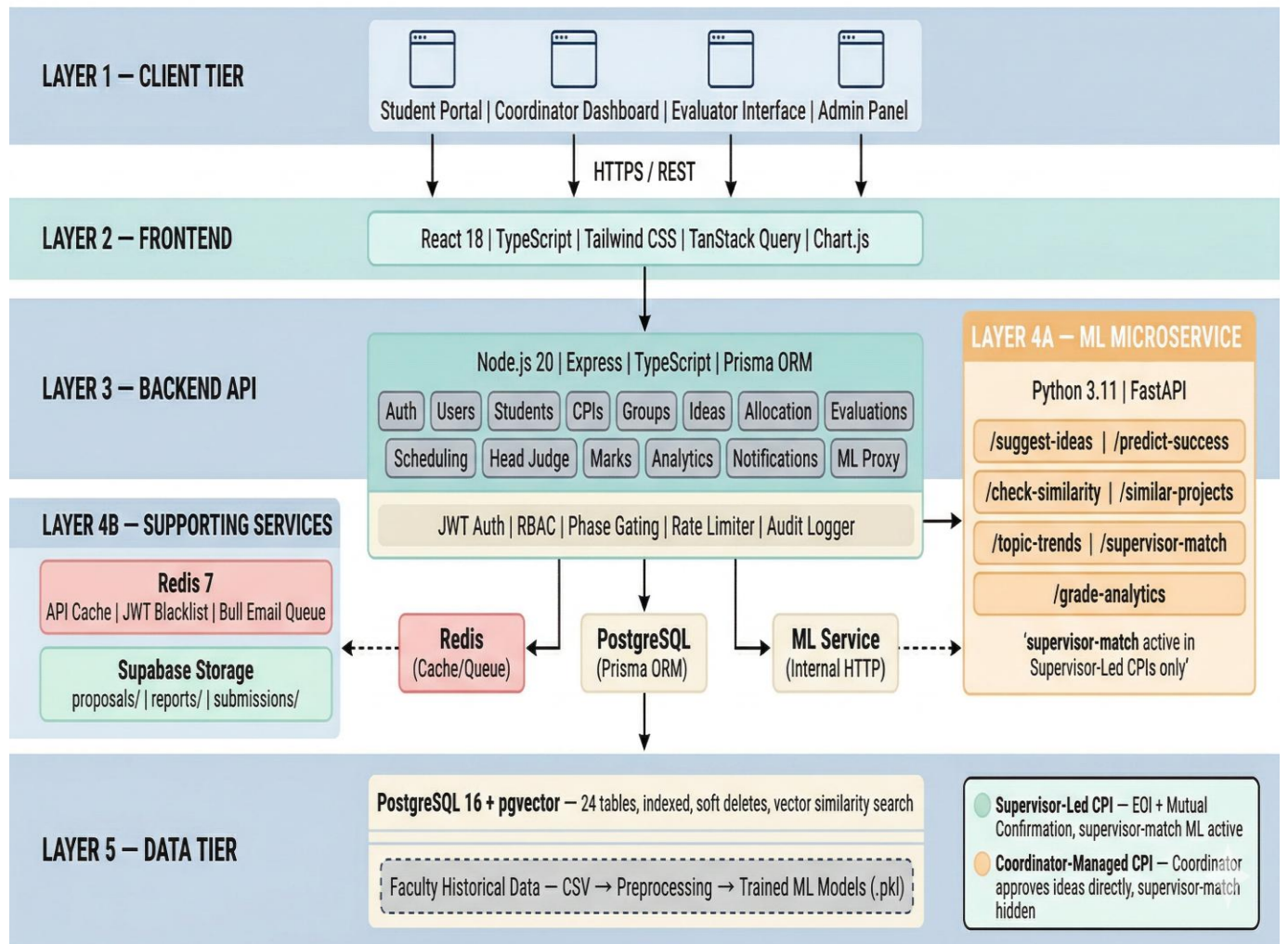


Figure 2: PSEMS Four-Tier System Architecture.

Role-Based Access Control (RBAC) Hierarchy

Level	Role	Scope	Primary Responsibilities	Key Restriction
1	Super Admin	System-wide	Full audit, all student & lecturer data, all CPIs, system config	Read-only on evaluation marks cannot modify
2	System Admin	System-wide	Student email DB management, bulk account creation, lecturer approval, assign coordinators	Cannot view individual rubric scores
3	Course Coordinator	Per CPI	Full CPI lifecycle timeline, mode determination, supervisors (optional), evaluators, rubrics, allocation, scheduling, analytics. In Coordinator-Managed mode: posts ideas directly and Accepts/Rejects student ideas.	Authority limited to assigned CPIs only
4a	Supervisor	Per CPI	Post ideas, mark willing to supervise student ideas, accept/decline group selections, supervise groups	Cannot score evaluations
4b	Evaluator	Per CPI	Submit rubric scores, customize session specifics, provide qualitative comments	Cannot see other evaluators' scores before Head Judge review
4c	Head Judge	Per CPI	Review all evaluator scores side-by-side, approve or request corrections, finalize marks to coordinator	Cannot directly change scores approve or flag only
5	Student	Per CPI	Register/form group, view supervisor ideas, post own idea (private), express interest, select project, submit deliverables, view own marks	Cannot see other groups' ideas or unpublished marks

6. Tools and Technologies

Backend & Database

Technology	Purpose & Justification
Node.js	High-performance non-blocking runtime handles concurrent requests from thousands of provisioned student accounts
Express.js + TypeScript	Modular HTTP framework with type safety across all business logic, RBAC middleware, and phase-gating logic
Prisma ORM	Type-safe database access with automatic migrations; parameterized queries prevent SQL injection
PostgreSQL	ACID-compliant relational database optimal for PSEMS's normalized multi-entity schema
pgvector	Native PostgreSQL extension for cosine similarity search on vector embeddings no separate vector DB needed
Redis	Sub-millisecond API caching, JWT token blacklisting, Bull queue for async email dispatch (student provisioning + notifications)
JWT + bcrypt	Stateless auth enabling horizontal scaling; bcrypt work factor 12; first-login password change enforced via token flag
Zod	Schema-first request validation prevents invalid data reaching business logic
Nodemailer	SMTP integration for automated student provisioning emails and all system notifications

Machine Learning Stack

Technology	Purpose & Justification
Python	Required runtime for scikit-learn and sentence-transformers ecosystem
FastAPI + uvicorn	Async-native Python API; superior concurrent performance vs Flask for ML inference serving
scikit-learn	Random Forest, K-Means, TF-IDF with comprehensive evaluation utilities (cross-validation, classification report)
sentence-transformers	Pre-trained SBERT all-MiniLM-L6-v2 84-dim semantic embeddings for similarity features
pandas + numpy	Data preprocessing, feature engineering, vector operations on historical dataset
NLTK	Tokenization, stop word removal, lemmatization for TF-IDF preprocessing pipeline
joblib	Trained model serialization to .pkl files for FastAPI startup loading
FAISS	Optional in-memory approximate nearest-neighbor search as backup to pgvector

Frontend & Infrastructure

Technology	Purpose
React 18 + TypeScript + Vite	Component-based UI with full type safety and fast development builds
Tailwind CSS + React Hook Form	Utility-first styling; performant form management with Zod integration
TanStack Query + Zustand	Server state caching/revalidation and lightweight global client state (session, notifications)
Chart.js / Recharts	Responsive chart components for coordinator analytics and grade distribution dashboards
Supabase Storage	Managed file storage for all student submissions with signed URL access control
Docker + GitHub Actions	Containerized deployment with automated CI/CD pipeline on merge to main
Vercel / Railway	Frontend CDN hosting; Node.js and Python ML service cloud deployment

7. Project Timeline

The implementation spans eight months (April–November 2026) across ten phases, following an iterative approach: core infrastructure first, parallel ML and frontend development tracks, converging in integrated testing before final submission.

Phase / Task	Apr		May		Jun		Jul		Aug		Sep		Oct		Nov	
1. Data Collection & Preprocessing																
2. System & DB Architecture Design																
3. Database Implementation																
4. Backend Core Development																
5. ML Model Training & Evaluation																
6. ML FastAPI Service Development																
7. Frontend Development																
8. ML-Backend Integration																
9. System Integration Testing																
10. Report Writing & Submission																

Figure 4: PSEMS Project Gantt Chart.

Phase	Period	Key Deliverable
1. Data Collection & Preprocessing	Apr – May	Cleaned historical dataset, preprocessing scripts, feature engineering notebook
2. System & DB Architecture Design	Apr – May	Finalized ER diagram, Prisma schema, API contract specification
3. Database Implementation	May – Jun	All PostgreSQL tables, pgvector index, student provisioning schema
4. Backend Core Development	May – Sep	All API modules, RBAC, phase gating, student email provisioning, selection mechanism
5. ML Model Training & Evaluation	May – Sep	Trained models with evaluation report (accuracy, F1, confusion matrix, silhouette scores)
6. ML FastAPI Service	Aug – Sep	All 7 ML endpoints deployed and unit tested
7. Frontend Development	Aug – Oct	All role-specific dashboards, lifecycle flow UIs, ML feature interfaces
8. ML-Backend Integration	Sep – Oct	End-to-end ML features functional with caching strategy
9. System Integration Testing	Oct – Nov	All user flows tested end-to-end; zero P1/P2 defects
10. Report Writing & Submission	Oct – Nov	Complete project report, documentation, final submission

8. Expected Outcomes

System Deliverables

Deliverable	Success Criterion
Fully functional PSEMS platform, all 7 roles, multiple concurrent CPIs, complete lifecycle	All 12 lifecycle steps operational with phase gating enforced
Automated student provisioning system bulk account creation + email dispatch	Correctly provisions 100+ student accounts and delivers credential emails
Dual-mode CPI system Supervisor-Led and Coordinator-Managed	Mode determined automatically at Step 3, correct idea visibility, approval flow, and selection mechanism applied per mode
Expression of Interest + Mutual Confirmation (Supervisor-Led mode)	Groups correctly matched; conflicts resolved coordinator override functional
Coordinator idea approval system (Coordinator-Managed mode)	Coordinator can Accept/Reject student ideas coordinator-posted ideas visible to all students
Fully customizable evaluation system with Head Judge verification	Coordinator can configure any rubric structure marks finalized only after Head Judge approval
ML microservice with 7 intelligent features supervisor compatibility scoring conditional on CPI mode	All 7 FastAPI endpoints responding supervisor match feature absent in Coordinator-Managed CPIs
Success predictor with evaluation report	Cross-validated accuracy $\geq 65\%$, F1-score ≥ 0.62 across grade bands
Plagiarism/similarity warning system	Correctly flags semantically similar ideas at cosine similarity ≥ 0.75 , false positive rate $< 15\%$
Coordinator analytical summary	Individual + aggregated marks displayed, all ML dashboard visualizations correct

Expected Benefits per Stakeholder

Stakeholder	Expected Benefit
Students	Automated LMS-style onboarding; transparent rubric marks; similarity warnings; ML-guided idea formulation; clear project selection mechanism
Course Coordinators	Full lifecycle control in one platform; est. 80% reduction in admin effort; complete analytical summary with aggregated marks
Supervisors	Structured idea posting; mutual confirmation for group selection; no unsolicited group assignments
Evaluators	Structured rubric interfaces; ability to customize session specifics; Head Judge verification ensuring consistency
Institution	Scalable reusable platform for all project-based courses; historical project knowledge preserved and made actionable

9. Conclusion

A complete plan for PSEMS, a centralized academic project management system of medium enterprise quality, is presented in this proposal created especially for Department of Computer Engineering, University of Sri Jayewardenepura.. It is based on the actual operational requirements of university project-based assessments. In addition to automating student provisioning in the manner of well-known platforms and formalizing supervisor-student matching through a useful Expression of Interest and Mutual Confirmation mechanism, the system gives the course coordinator complete control over the project lifecycle, from CPI creation and timeline definition to final analytical summary.

The platform addresses four documented institutional problems: fragmented student onboarding, unstructured supervisor-student allocation, inconsistent evaluation without verification, and unused historical project data. The Machine Learning and Data Mining layer transforms this historical data into seven actionable intelligent features that guide students, supervisors, and coordinators at every stage of the project lifecycle from idea formulation through to final results.

Technically, the job can be completed within the suggested time frame. Every technology is established, thoroughly documented, and freely available. With automated provisioning for thousands of students, scalable database indexing, decoupled machine learning microservices, and a role structure that clearly divides responsibilities among all system users, the architecture is built for university-scale deployment from the start. When PSEMS is finished, it will provide a documented, tested, and working system with a clear route to faculty-wide deployment.

10. References

- [1] M. Al-Ajlan and H. Zedan, "Why Moodle," in Proc. 12th IEEE Int. Workshop Future Trends Distrib. Comput. Syst. (FTDCS 2008), Kunming, China, Oct. 2008, pp. 58–64, doi: 10.1109/FTDCS.2008.22.
- [2] K. Siau and M. Rossi, "Evaluation techniques for systems analysis and design modelling methods: A review and comparative analysis," *Inf. Syst. J.*, vol. 21, no. 3, pp. 249–268, May 2011, doi: 10.1111/j.1365-2575.2007.00255.x.
- [3] D. Gale and L. S. Shapley, "College admissions and the stability of marriage," *Am. Math. Monthly*, vol. 69, no. 1, pp. 9–15, Jan. 1962, doi: 10.2307/2312726.
- [4] R. W. Irving, D. F. Manlove, and S. Scott, "The stable marriage problem with master preference lists," *Discrete Appl. Math.*, vol. 156, no. 15, pp. 2959–2977, Aug. 2008, doi: 10.1016/j.dam.2008.01.003.
- [5] C. Romero and S. Ventura, "Educational data mining: A review of the state of the art," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 40, no. 6, pp. 601–618, Nov. 2010, doi: 10.1109/TSMCC.2010.2053532.
- [6] R. S. J. d. Baker and K. Yacef, "The state of educational data mining in 2009: A review and future visions," *J. Educ. Data Min.*, vol. 1, no. 1, pp. 3–17, Oct. 2009.
- [7] E. A. Amrieh, T. Hamtini, and I. Aljarah, "Mining educational data to predict student's academic performance using ensemble methods," *Int. J. Database Theory Appl.*, vol. 9, no. 8, pp. 119–136, 2016, doi: 10.14257/ijda.2016.9.8.13.
- [8] S. M. Alzahrani, N. Salim, and A. Abraham, "Understanding plagiarism linguistic patterns, textual features, and detection methods," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 42, no. 2, pp. 133–149, Mar. 2012, doi: 10.1109/TSMCC.2011.2134847.
- [9] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988, doi: 10.1016/0306-4573(88)90021-0.
- [10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT 2019, Minneapolis, MN, USA, Jun. 2019, pp. 4171–4186, doi: 10.18653/v1/N19-1423.
- [11] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP 2019, Hong Kong, China, Nov. 2019, pp. 3982–3992, doi: 10.18653/v1/D19-1410.
- [12] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, Jul. 2021, doi: 10.1109/TBDATA.2019.2921572.
- [13] S. Thuseethan, S. Achchuthan, and S. Kuhanesan, "Usability evaluation of learning management systems in Sri Lankan universities," *Global J. Comput. Sci. Technol.*, vol. 15, no. 1, pp. 1–7, 2015.
- [14] S. Thuseethan, "Department management system for departments of Sri Lankan universities," *Int. J. Sci. Technol. Res.*, vol. 3, no. 6, pp. 173–175, Jun. 2014.
- [15] D. N. T. Gunawardhana, C. D. Jayawardena, and R. A. R. C. Gopura, "Readiness for 'Social Learning Management System' (Social LMS) in Sri Lankan universities: A case study research based on students' perspectives," in Proc. Int. Conf. Adv. ICT Educ. (ICATE 2018), Colombo, Sri Lanka, 2018.